

# Chess Engine: Technical Report

## 1 Overview

This project presents a fully functional chess engine capable of evaluating positions, exploring move sequences, and selecting optimal continuations through classical search algorithms, including a time-management system.

The implementation builds on the `python-chess` library, which provides data structures for representing the chessboard, moves, and game state. It also handles rule enforcement (e.g., legality of moves, detection of checkmate and stalemate, move notation, and position hashing), allowing the project to focus on the higher-level algorithmic components of a chess engine.

## 2 Static board evaluation

The board evaluation function is the core analytical component of the engine. Its purpose is to assign a numerical score to a given (static) chess position, indicating which side is favored and by how much. The score is expressed in *centipawns*, where one pawn corresponds to a value of 100. Following chess conventions, a positive score indicates an advantage for White, while a negative score indicates an advantage for Black.

The evaluation is based on two components: material balance and positional advantage. The former is given by a simple sum of the values of the pieces that each side still has on the board. These base values, drawn from classical chess programming, are intended to reflect the average relative strength of each piece:

$$\text{Pawn} = 100, \quad \text{Knight} = 320, \quad \text{Bishop} = 330, \quad \text{Rook} = 500, \quad \text{Queen} = 900.$$

The positional component refines this raw evaluation by introducing context-dependent adjustments based on piece placement and structural features of the position. It combines information from two main sources: *piece-square tables* (PSTs) and heuristic bonuses/penalties.

PSTs are 8×8 numerical tables that encode the desirability of each square for each piece type. During evaluation, every piece contributes its base value plus the bonus (or penalty) corresponding to its square. For Black, the tables are mirrored vertically to maintain symmetry.

For example, knights are rewarded for occupying central squares (values in centipawns):

$$\text{KNIGHT\_MG\_TABLE} = \begin{bmatrix} -50 & -40 & -30 & -30 & -30 & -30 & -40 & -50 \\ -40 & -20 & 0 & 5 & 5 & 0 & -20 & -40 \\ -30 & 5 & 10 & 15 & 15 & 10 & 5 & -30 \\ -30 & 0 & 15 & 20 & 20 & 15 & 0 & -30 \\ -30 & 5 & 15 & 20 & 20 & 15 & 5 & -30 \\ -30 & 0 & 10 & 15 & 15 & 10 & 0 & -30 \\ -40 & -20 & 0 & 0 & 0 & 0 & -20 & -40 \\ -50 & -40 & -30 & -30 & -30 & -30 & -40 & -50 \end{bmatrix}$$

Intuitively, the preferred placement of a piece can change substantially as the game progresses. The king provides a clear example: during the middlegame, keeping it away from the center of the board generally improves its safety, whereas in the endgame an active, centralized king often plays a decisive role.

For this reason, as of version 1.1, the engine computes a game-phase variable  $\psi \in [0, 1]$  for each evaluated position based on the weighted amount of non-pawn material remaining on the board, where lower values of  $\psi$  correspond to late-game stages. The engine then uses linear interpolation to introduce phase dependence into the PSTs:

$$\text{PST} = \psi \text{PST}_{\text{Middlegame}} + (1 - \psi) \text{PST}_{\text{Endgame}}.$$

Heuristic features further refine this evaluation by applying bonuses and penalties for factors such as bishop pairs, rook coordination, and pawn structure features such as passed, doubled and isolated pawns.

### 3 Move ordering

Before analyzing possible continuations, the engine assigns a priority score to each legal move and sorts them from most to least promising. Each move's score is based on simple heuristics (captures, promotions, checks).

The resulting ordered list does not determine which move is best, but rather establishes a ranking of candidate moves for deeper analysis in the search phase. This becomes vital when the engine begins to search through possible continuations: as described in the next section, the algorithm can often skip large parts of its search if it encounters strong moves early. By examining the most promising moves first, the engine is more likely to find good positions quickly, increasing computational efficiency.

### 4 Move search and alpha-beta pruning

While the evaluation function measures the quality of a single position, a chess engine must also reason about how the position might evolve after a sequence of moves. This is achieved through a search algorithm which simulates future play by exploring possible move sequences for both sides.

At a conceptual level, the program constructs a search tree, where each node represents a board position and each branch represents a possible move. White tries to maximize the score (find the best position for itself), while Black tries to minimize it (reduce White's advantage). This idea forms the basis of the minimax algorithm.

To make the search more efficient, the engine uses *alpha-beta pruning*, a technique that avoids exploring branches that cannot influence the final decision. It works by keeping track of two limits (alpha and beta) as the engine explores the tree of moves, where alpha represents the best score that the player to move can guarantee so far, and beta represents the best score that the opponent can force. As the search progresses, alpha and beta define a range of outcomes that are still worth considering: if the engine realizes that a line of play cannot possibly lead to a better result than one already found, it can safely ignore the rest of that branch.

## 5 Quiescence search

When the main search function reaches its maximum depth, it stops looking further ahead and asks for an evaluation of the current position. However, this approach can be unreliable if the position is tactically unstable (for example, if there are captures or checks that could immediately change the material balance). To handle this, instead of evaluating the position directly, the engine calls the *quiescence search* function (`qsearch`), which continues exploring only “noisy” moves.

This allows the engine to “tidy up” the evaluation at the edges of the search tree. Once the tactical noise has settled, the resulting score is passed back to the main search, giving a much more stable and realistic assessment of the position and avoiding the so-called horizon effect.

## 6 Transposition table caching

As the engine analyzes possible continuations, it often encounters the same position multiple times through different sequences of moves. Re-evaluating such positions repeatedly would be redundant and inefficient. To prevent this, the engine maintains a *transposition table* (TT), a form of memory cache that stores information about previously analyzed positions. When the engine reaches a new position, it first checks whether useful information about that position already exists in the table. When the stored entry satisfies the requirements of the current search, this information can be reused rather than computed again from scratch.

Each board position is identified by a Zobrist hash, a 64-bit number that acts as a unique fingerprint for that specific arrangement of pieces. The process is managed by two helper functions: `tt_store` and `tt_probe`.

After finishing the analysis of a position, the function `tt_store` saves the result into the table. It records the position’s hash, the search depth, the best move found, and a flag describing how the stored score should be interpreted. The flag distinguishes whether the stored score is an exact evaluation, a lower bound (score  $\geq \beta$ ), or an upper bound (score  $\leq \alpha$ ), allowing the alpha-beta search to reuse information safely.

The function `tt_probe` checks whether the current position is already present in the transposition table. A stored value is only reused when the entry was computed to a sufficient search depth and its bound is compatible with the current alpha-beta window. Exact entries can be returned directly, while lower- and upper-bound entries are used when they imply a valid cutoff.

## 7 Iterative deepening

In the initial release, move selection was performed at a fixed search depth. While this provides a simple and reproducible search procedure, its computational cost can vary substantially across positions: a quiet position with relatively few legal continuations may be searched quickly, whereas a tactically complex position at the same nominal depth may require considerably more work.

Version 1.1 therefore introduces *iterative deepening*. Instead of immediately searching to a predetermined depth, the engine successively performs searches at depths 1, 2, 3, ... until the

available thinking time expires. Although this appears to repeat previous work, earlier iterations provide useful information for deeper searches. In particular, the best move from the previous iteration is searched first at the next depth, which increases the effectiveness of alpha-beta pruning.

The engine currently supports both fixed-depth and time-controlled modes. Fixed-depth search is retained for reproducible testing, while timed search is used for practical play. Through the UCI interface, the engine recognizes commands such as `go depth` and `go movetime`. It can also allocate its own thinking time from the remaining game clock using `wtime`, `btime`, `winc`, `binc`, and `movestogo`.

## 8 Time management system

For clock-based play, the engine adjusts its thinking time in a way that resembles human decision-making. When plenty of time remains on the clock, it can afford to spend longer analyzing the current position and search more deeply. As the available time decreases, it becomes progressively more conservative and makes decisions more quickly to avoid running out of time.

To achieve this, the engine keeps a small portion of the remaining clock as a safety reserve and distributes the rest across the moves it expects to play. Any time increment received after each move is also taken into account. The resulting time allowance is then passed to the iterative-deepening search, so the engine naturally searches deeper when more thinking time is available and stops earlier when under pressure.

## 9 Search instrumentation

To make the behavior of the search algorithm observable, version 1.1 introduces a set of diagnostic counters. For each root search, the engine records the number of regular search nodes and quiescence nodes, transposition-table probes and usable hits, alpha-beta and quiescence cutoffs, elapsed computation time, nodes searched per second (NPS), and the deepest fully completed iteration.

These statistics are displayed directly in the command-line interface and can also be reported through UCI-compatible `info` messages. Besides assisting with debugging, the instrumentation provides a quantitative basis for evaluating future search optimizations, since changes in move ordering or pruning can be compared in terms of search-tree size and computational cost rather than only by observing game outcomes.

## 10 Weaknesses and potential future upgrades

While the engine demonstrates reliable functionality and can autonomously play complete games without external input, it remains a simplified implementation designed for transparency and experimentation. The current design prioritizes readability and conceptual clarity over raw playing strength or computational efficiency, making it an effective tool for illustrating classical search and evaluation techniques.

Several areas for improvement can be identified:

- Search efficiency could be improved through more advanced move-ordering heuristics. In particular, killer-move and history heuristics could prioritize quiet moves that have previously produced alpha-beta cutoffs, increasing the probability of pruning unpromising branches earlier.
- More aggressive selective-search techniques, such as null-move pruning, could substantially reduce the size of the search tree. Such methods would require careful testing because, unlike move ordering, they intentionally avoid fully searching some branches and may therefore introduce position-dependent errors.
- From an engineering perspective, further performance gains could be obtained through lower-level optimization of board representation, move generation, evaluation, and memory management. These changes would trade some of the current implementation's transparency for greater computational speed.
- Additional features such as an opening book or endgame tablebases could further improve practical playing strength and realism.

See the project's changelog to view version history, updates, and WIP / planned features.